10P SHINE
INTERNSHIP PROGRAM

Project Report: Karachi AQI Forecaster

Organization: 10Pearls
Version: 1.0
Date: 18-02-2026
Document: Project Report: Karachi AQI Forecaster

Prepared by: Hafiz Salahuddin
Mentored by: Muhammad Mobeen

Table of Contents

1. Executive Summary 3
2. System Architecture & Infrastructure Pivot 3
3. Data Ingestion & Exploratory Data Analysis (EDA) 4
   1.1 Data Preprocessing & Transformation 4
   1.2 Temporal Trend Identification (Business Context) 5
4. Predictive Modeling & Explainability 7
5. Technical Challenges & Engineering Solutions 8
   1.3 The "Crystal Ball" Problem (Look-Ahead Bias) 8
   1.4 Data Leakage in Feature Engineering 8
   1.5 LSTM "Dead Neurons" (Flatline Predictions) 9
   1.6 Forecast Disconnect ("The Drop") 9
   1.7 Static Forecast Visuals 9
   1.8 Database & Timezone Synchronization 9
   1.9 Model Artifact Mismatches 10
6. Automation & Final Deliverables 10

Table of Figures

Figure 1- Architecture Diagram 3
Figure 2(6 Month istorical View) 5
Figure 3 (Hourly Cycles of AQI) 6
Figure 4 (weekday & weekend bar chart of AQI) 6
Figure 5 Heat Map 7

1. Executive Summary

The Karachi AQI Forecaster is an end-to-end Machine Learning prediction service designed to forecast the Air Quality Index (AQI) up to 3 days in advance. The project fulfills the requirement of utilizing a 100% serverless stack. It integrates automated data ingestion, advanced feature engineering, multiple predictive models (Statistical and Deep Learning), and an interactive web dashboard for real-time monitoring and hazardous condition alerts.

2. System Architecture & Infrastructure Pivot

The system was designed to be fully automated and serverless.

The Hopsworks to Supabase Pivot: Initially, the pipeline was architected to utilize Hopsworks as the primary Feature Store. However, we encountered persistent infrastructure instability and integration roadblocks, even after upgrading to a paid tier. After consulting with the project mentor and receiving official approval, we successfully pivoted the architecture to Supabase. Supabase proved to be an exceptionally reliable serverless alternative, seamlessly acting as both our real-time Feature Store (PostgreSQL) and Model Registry (Object Storage).

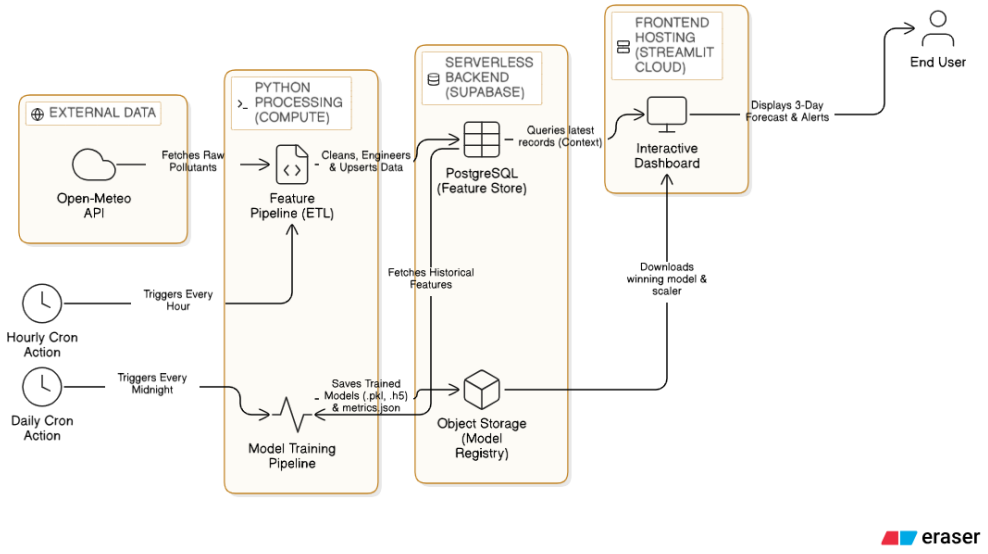

Figure 1- Architecture Diagram

Core Stack:

- **Data Source:** Open-Meteo Air Quality API
- **Database & Registry:** Supabase
- **CI/CD Automation:** GitHub Actions (Hourly/Daily Cron Jobs)
- **Modeling:** Scikit-Learn, XGBoost, TensorFlow/Keras, SHAP
- **Frontend UI:** Streamlit Community Cloud

3. Data Ingestion & Exploratory Data Analysis (EDA)

In alignment with the project guidelines, EDA was performed comprehensively on Google Colab to ensure visualizations were clear and understandable for non-technical stakeholders. We retrieved 180 days (6 months) of historical hourly data.

1.1 Data Preprocessing & Transformation

Following academic and enterprise best practices, the raw data underwent rigorous cleaning:

- **Missing Value Imputation:** Because air quality is a continuous temporal event, dropping missing rows would break the sequential memory needed for lag features. We utilized Linear Interpolation to smoothly estimate missing values based on surrounding hours.
- **Noise Smoothing:** Applied a 3-hour moving average to the raw input features to filter out white noise while preserving the true underlying pollutant trends.
- **Outlier Capping (Winsorization):** Standard outlier removal is destructive to time-series data. Instead, we used the Interquartile Range (IQR) method. Values exceeding the $1.5 \times IQR threshold were capped at the upper boundary.
- **Statistical Transformations:** We conducted a skewness analysis across all 7 pollutants. Variables that exhibited highly right-skewed distributions (Skewness > 1.0), specifically Nitrogen Dioxide (NO_2) and Dust, were treated with a Logarithmic Transformation (np.log1p(x)) to compress massive variations and stabilize variance.

1.2 Temporal Trend Identification (Business Context)

To ensure the project findings were accessible to non-technical stakeholders and management, we generated clear visual trend analyses to understand the actual pollution dynamics in Karachi.

1. Macro Trend (6-Month Historical View)

Plotting the continuous AQI revealed the broader seasonal drift. We overlaid a static "Unhealthy Threshold" line (AQI = 100), which instantly highlighted that certain months had a much higher frequency of hazardous days.

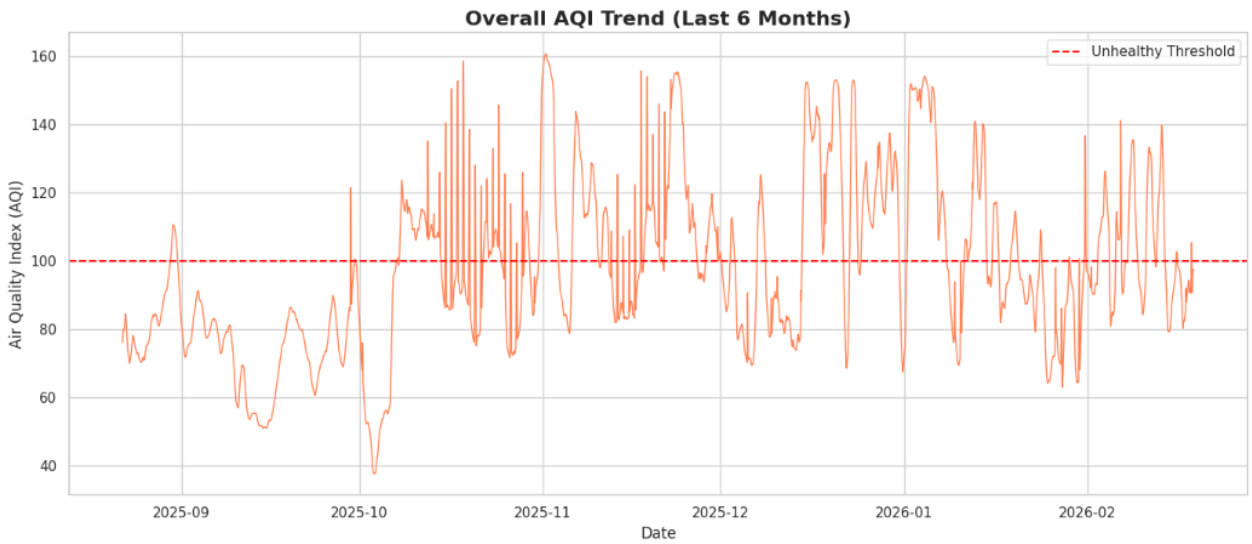

Figure 2(6 Month istorical View)

2. Diurnal Rhythm (Hourly Cycles)

By aggregating the average AQI by the hour of the day, a distinct bimodal (two-peak) pattern emerged. Pollution levels reliably spiked during the morning commute and evening rush hours, mathematically justifying the inclusion of hour as a primary feature for the predictive models.

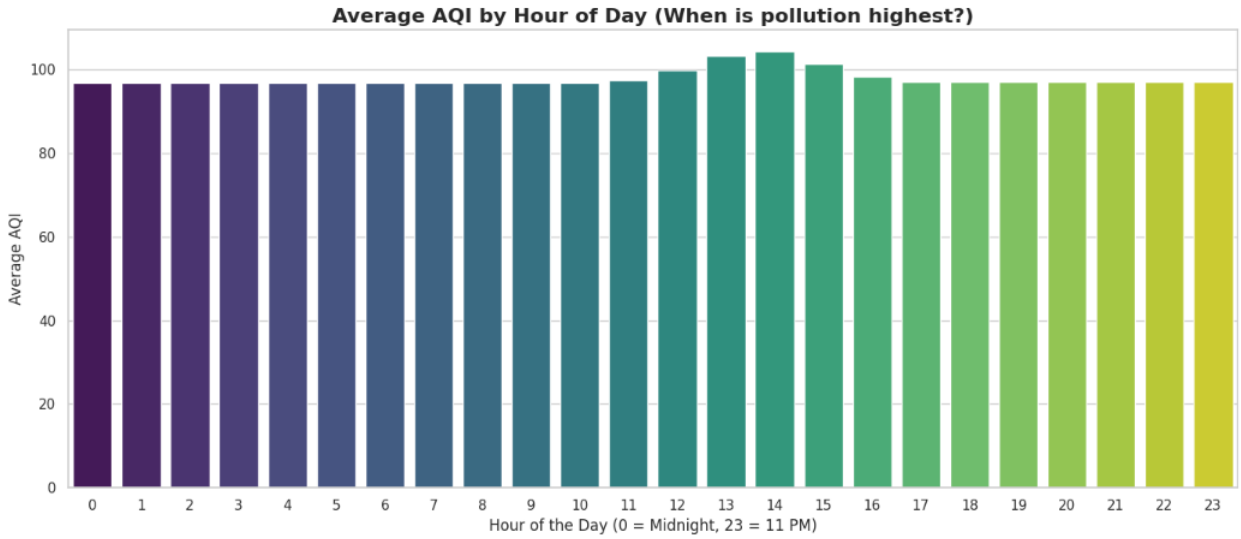

Figure 3 (Hourly Cycles of AQI)

3. Weekly Seasonality (Weekday vs. Weekend)

Aggregating AQI by day_of_week showed a noticeable drop in average particulate matter on Sundays compared to mid-week peaks. This confirmed the impact of commercial/industrial activity and weekday traffic.

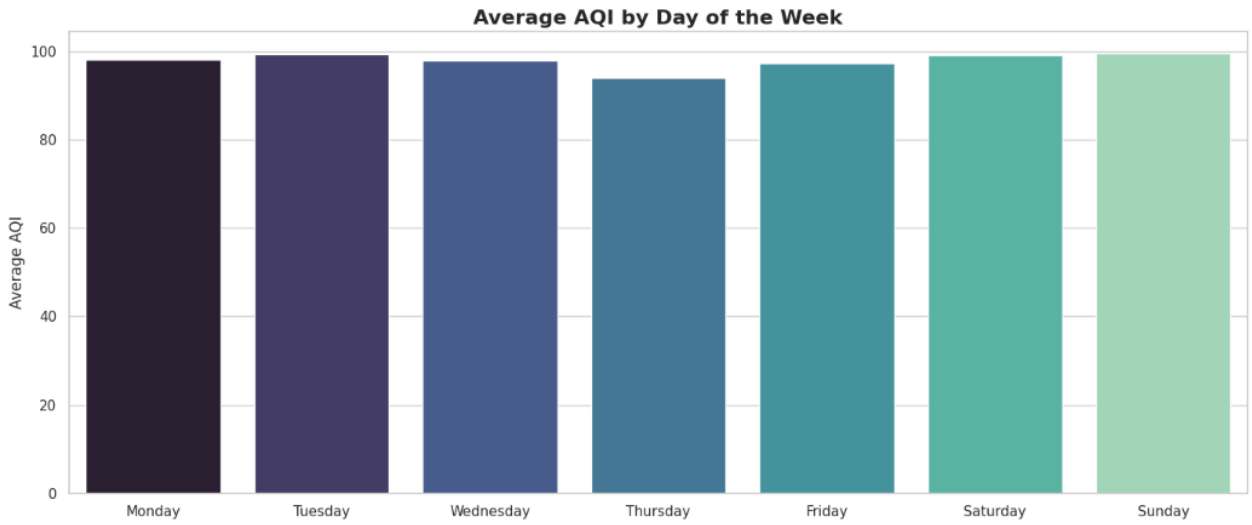

Figure 4 (weekday & weekend bar chart of AQI)

4. Feature Correlation & Dimensionality

Before training, we generated a Feature Correlation Heatmap to check for multicollinearity. While PM2.5 and PM10 were highly correlated, Tree-based models are robust to this, so we retained both. Crucially, the correlation analysis confirmed that our engineered derivative feature (aqi_change_rate) had a unique, non-redundant relationship with the target variable.

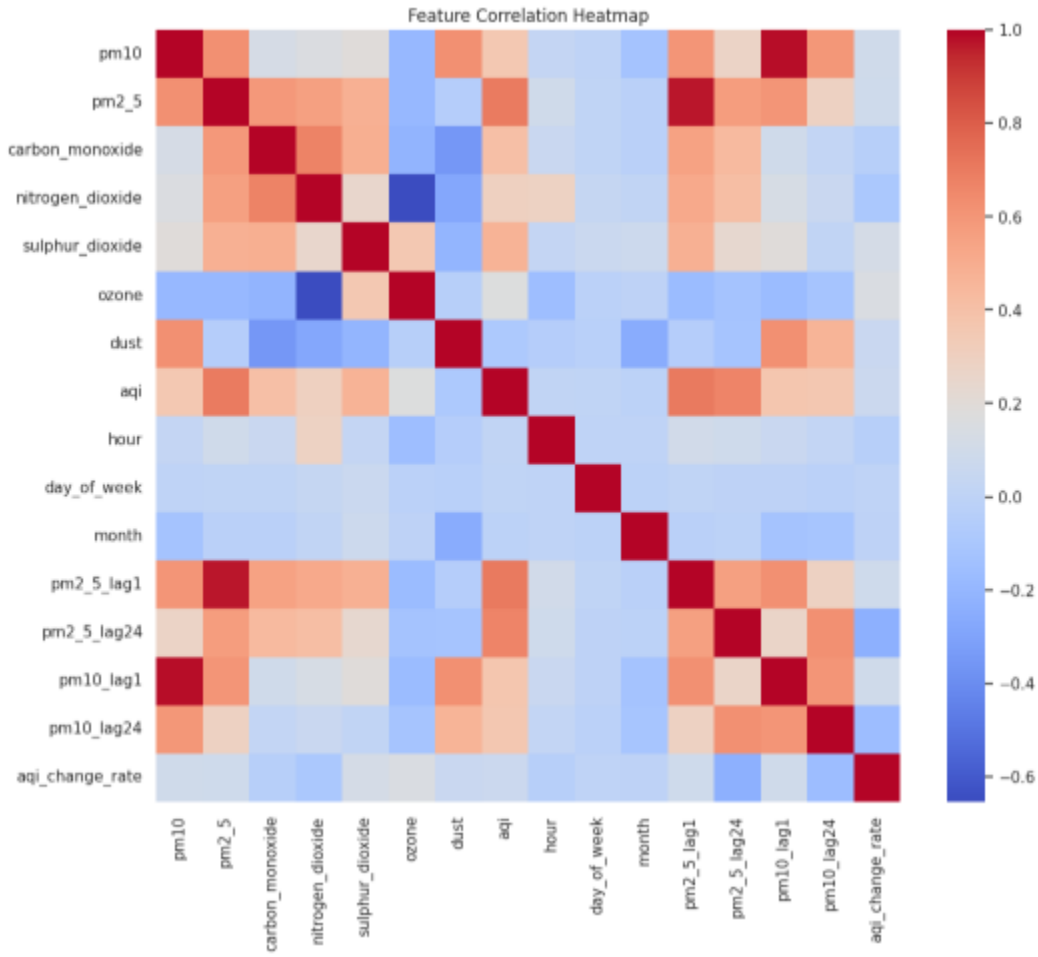

Figure 5 Heat Map

4. Predictive Modeling & Explainability

To satisfy the requirement of comparing various architectures, three distinct models were trained and evaluated:

1. **Random Forest Regressor:** Acted as our robust statistical baseline.
2. **XGBoost Regressor:** Utilized for advanced, gradient-boosted tree performance.
3. **LSTM (Deep Learning):** Implemented using TensorFlow to evaluate sequential pattern recognition.

Model Explainability: We integrated SHAP (Shapley Additive exPlanations) to ensure the UI and model were fully explainable to end-users. The dashboard visualizes feature importance, allowing users to see exactly which pollutants (e.g., PM2.5 vs. PM10) are driving the current AQI forecast.

5. Technical Challenges & Engineering Solutions

During the development lifecycle, several complex MLOps and Data Science challenges were encountered and systematically resolved:

1.3 The "Crystal Ball" Problem (Look-Ahead Bias)

- **Issue:** Initially, the model was trained to predict the *current* AQI based on *current* weather conditions. However, in a real-world forecast scenario, we do not have future weather data. This forced the dashboard to use static averages for future inputs, resulting in flat, unrealistic predictions.
- **Solution:** We implemented **Lead Time Forecasting**. In the training pipeline, we shifted the target variable by -24 hours (df['aqi'].shift(-24)). This forced the model to learn the relationship between *today's* weather and *tomorrow's* air quality, enabling true predictive capability without needing future data.

1.4 Data Leakage in Feature Engineering

- **Issue:** The aqi_change_rate feature was originally calculated as current_aqi - previous_aqi. During training, this inadvertently gave the model access to the target variable (current AQI), resulting in artificially high accuracy scores that collapsed in production.
- **Solution:** We corrected the logic to use strictly historical data: df['aqi'].shift(1).diff(). This ensures the model only sees the change rate leading up to the *previous* hour, eliminating leakage.

1.5 LSTM "Dead Neurons" (Flatline Predictions)

- **Issue:** The Deep Learning (LSTM) model initially predicted a straight line of zeros. This was caused by using the ReLU activation function (which causes the "dying ReLU" problem in RNNs) and unscaled target variables causing gradient explosions.
- **Solution:** Switched the LSTM layer activation to tanh (the mathematical standard for RNNs) and implemented a dedicated MinMaxScaler for the target variable, scaling it to [0, 1] during training and inverse-transforming predictions for the dashboard.

1.6 Forecast Disconnect ("The Drop")

- **Issue:** Because the model predicts 24 hours ahead, its prediction for "right now" is based on data from yesterday. If weather patterns changed suddenly, the forecast line would start at a value significantly different from the actual current AQI, creating a visual "jump".
- **Solution:** We implemented **Anchoring (Bias Correction)**. The system calculates the difference between the model's prediction for the current hour and the *actual* current AQI. This bias is applied to the forecast curve and decayed over time, ensuring seamless connection to historical data.

1.7 Static Forecast Visuals

- **Issue:** Even with a good model, the 3-day forecast looked robotic because the inputs (future weather) were unknown and estimated as flat averages.
- **Solution:** We implemented **Pattern Projection & Organic Variation**. We repeated the last 24 hours of weather data into the future to preserve diurnal cycles (rush hour peaks) and added a random walk (slight noise) to simulate changing weather fronts, resulting in a dynamic, realistic forecast.

1.8 Database & Timezone Synchronization

- **Issue:** The dashboard occasionally displayed old data or incorrect times due to UTC defaults and Supabase query limits.

- **Solution:** Updated db_client.py to explicitly order by timestamp descending and fetch the latest rows. Enforced the Asia/Karachi timezone across the entire ETL, Training, and Dashboard pipelines to ensure peak hours aligned with local reality.

1.9 Model Artifact Mismatches

- **Issue:** Adding new features (like rolling_7d) caused the dashboard to crash because the saved models expected the old feature set.
- **Solution:** Implemented robust error handling to catch ValueError mismatches and added a "🔄 Reload Models" action to clear the Streamlit cache and automatically re-download the latest artifacts from the Model Registry.

6. Automation & Final Deliverables

CI/CD Pipelines: GitHub Actions were configured to automatically run the Feature Pipeline every hour (fetching new API data) and the Model Training Pipeline every day at midnight (retraining models on fresh data).

Web Application: A Streamlit dashboard was successfully deployed, fulfilling the requirement for a clear, interactive UI. It features historical trend graphs, real-time predictions, SHAP explainability, and critical visual alerts for Hazardous AQI levels (>300).